

编译原理

3. 语法分析-自底向上

rainoftime.github.io
浙江大学
计算机科学与技术学院

课程内容

1. Introduction
2. Lexical Analysis
- 3. Parsing**
4. Abstract Syntax
5. Semantic Analysis
6. Activation Record
7. Translating into Intermediate Code
8. Basic Blocks and Traces
9. Instruction Selection
10. Liveness Analysis
11. Register Allocation
13. Garbage Collection
14. Object-oriented Languages
18. Loop Optimizations

回顾: SLR(1)的问题/局限

- SLR技术解决冲突的方法
 - 按照 $A \rightarrow \alpha$ 进行归约的条件：下一个输入符号 x 在某个句型中跟在 A 之后，也就是 $x \in \text{Follow}(A)$
- “设定”：LR分析过程是最右推导的逆
 - 每一步都应该是最右句型
 - 如果 βAx 不是任何最右句型的前缀，那么即使 x 在某个句型中跟在 A 之后，也不应该按 $A \rightarrow \alpha$ 归约
 - （利用Follow集可看作“可能不够精确的近似”）

进行归约的条件更加严格才能进一步降低冲突的可能

本讲内容

1

LR(1) 分析

2

LALR(1) 分析

3

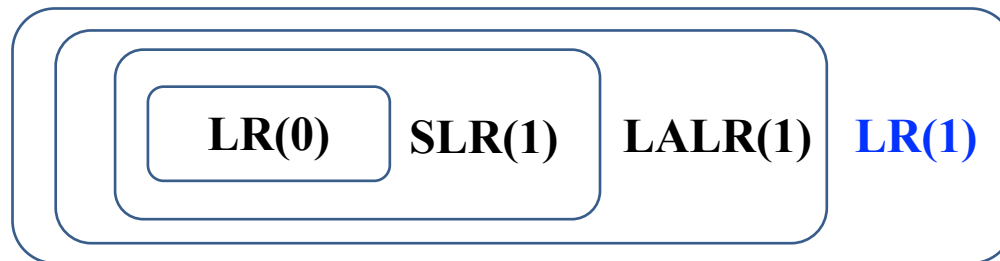
Parser Generator

4

语法分析小结

5

错误恢复



1. LR(1)分析

LR(1)项 (LR(1) Items)

- LR(1)项：包含更多信息来消除一些归约动作
- 实际的做法相当于“分裂”一些LR(0)状态，精确指明何时应该归约
- LR(1)项的形式 $A \rightarrow \alpha \bullet \beta, a$
 - a 称为向前看符号，可以是终结符号或者\$
 - sequence α is on top of the (symbol) stack
 - the head of the input is a string derivable from βa

关键区别：LR(1) 项 = LR(0) 项 + 一个向前看符号 a 。
归约时只有当前输入 token 恰为 a 才进行归约

LR(1) Parsing: Closure

Closure(I) = LR(1)
repeat
 for any item $(A \rightarrow \alpha \bullet X\beta, z)$ in I
 for any production $X \rightarrow \gamma$
 for any $w \in \text{First}(\beta z)$
 $I \leftarrow I \cup \{(X \rightarrow \bullet \gamma, w)\}$
until I does not change
return I

Closure(I) = LR(0)
repeat
 for any item $A \rightarrow \alpha \bullet X\beta$ in I
 for any production $X \rightarrow \gamma$
 $I \leftarrow I \cup \{X \rightarrow \bullet \gamma\}$
until I does not change.
return I

- 起始状态: LR(1)项 $S' \rightarrow \bullet S \$, ?$ 的闭包
 - ”?” 是什么无关紧要, 因为\$不会被移进

LR(1) Parsing: Closure

Closure(I) = LR(1)
repeat
 for any item $(A \rightarrow \alpha \bullet X\beta, z)$ in I
 for any production $X \rightarrow \gamma$
 for any $w \in \text{First}(\beta z)$
 $I \leftarrow I \cup \{(X \rightarrow \bullet \gamma, w)\}$
 until I does not change
return I

Closure(I) = LR(0)
repeat
 for any item $A \rightarrow \alpha \bullet X\beta$ in I
 for any production $X \rightarrow \gamma$
 $I \leftarrow I \cup \{X \rightarrow \bullet \gamma\}$
 until I does not change.
return I

- 处理 ϵ -transitions(即添加 $X \rightarrow \bullet \gamma$)时 ,
 - 记录向前看符号 w
 - $w \in \text{First}(\beta z)$: 把 $A \rightarrow \alpha \bullet X\beta, z$ 的信息“传递”到 $X \rightarrow \bullet \gamma, w$
- 含义: $A \rightarrow \alpha \bullet X\beta, z$: 表示输入头是可从 $X\beta z$ 推导的串
- 新加的item $X \rightarrow \bullet \gamma, w$: 表示输入头是可从 γw 推导的串

例: LR(1) Closure

```
Closure(I) =  
  repeat  
    for any item  $(A \rightarrow \alpha \bullet X\beta, z)$  in  $I$   
      for any production  $X \rightarrow \gamma$   
        for any  $w \in \text{First}(\beta z)$   
           $I \leftarrow I \cup \{(X \rightarrow \bullet \gamma, w)\}$   
    until  $I$  does not change  
  return  $I$ 
```

$S' \rightarrow \cdot S \$$?

0 $S' \rightarrow S \$$

1 $S \rightarrow V = E$

2 $S \rightarrow E$

3 $E \rightarrow V$

4 $V \rightarrow x$

5 $V \rightarrow * E$

Closure($\{S' \rightarrow \cdot S \$, ?\}$) = ??

从 $S' \rightarrow \cdot S \$, ?$ 开始, 对每个 $A \rightarrow \alpha \bullet X\beta, z$, 为 X 的每条产生式 $X \rightarrow \gamma$, 添加 $(X \rightarrow \bullet \gamma, w)$, 其中 $w \in \text{First}(\beta z)$

例: LR(1) Closure

```
Closure(I) =  
  repeat  
    for any item  $(A \rightarrow \alpha \bullet X\beta, z)$  in  $I$   
      for any production  $X \rightarrow \gamma$   
        for any  $w \in \text{First}(\beta z)$   
           $I \leftarrow I \cup \{(X \rightarrow \bullet \gamma, w)\}$   
    until  $I$  does not change  
  return  $I$ 
```

0 $S' \rightarrow S \$$
1 $S \rightarrow V = E$
2 $S \rightarrow E$
3 $E \rightarrow V$
4 $V \rightarrow x$
5 $V \rightarrow * E$

$S' \rightarrow \bullet S \$$?

$S \rightarrow \bullet V = E$ \$

$S \rightarrow \bullet E$ \$

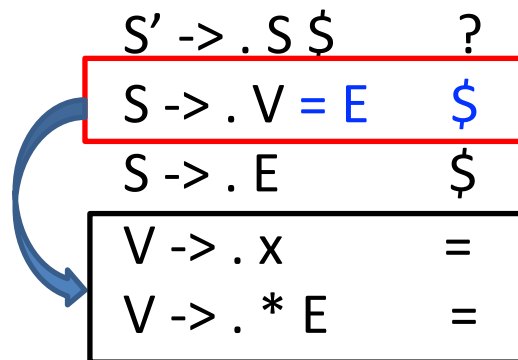
$\text{First}(\$?): \{\$ \}$

1. 首先选择S来生成新的item. 此时 β 相当于\$, z 是“?”, 因此 βz 是\$?

例: LR(1) Closure

```
Closure(I) =  
  repeat  
    for any item  $(A \rightarrow \alpha \bullet X\beta, z)$  in  $I$   
      for any production  $X \rightarrow \gamma$   
        for any  $w \in \text{First}(\beta z)$   
           $I \leftarrow I \cup \{(X \rightarrow \bullet \gamma, w)\}$   
    until  $I$  does not change  
  return  $I$ 
```

0 $S' \rightarrow S \$$
1 $S \rightarrow V = E$
2 $S \rightarrow E$
3 $E \rightarrow V$
4 $V \rightarrow x$
5 $V \rightarrow * E$



$\text{First}(=E\$): \{=\}$

2. 继续选择V来生成新的item. 此时 β 是 $=E$, z 是 $\$$, 因此 βz 是 $=E\$$

例: LR(1) Closure

```
Closure(I) =  
  repeat  
    for any item  $(A \rightarrow \alpha \bullet X\beta, z)$  in  $I$   
      for any production  $X \rightarrow \gamma$   
        for any  $w \in \text{First}(\beta z)$   
           $I \leftarrow I \cup \{(X \rightarrow \bullet \gamma, w)\}$   
    until  $I$  does not change  
  return  $I$ 
```

0 $S' \rightarrow S \$$
1 $S \rightarrow V = E$
2 $S \rightarrow E$
3 $E \rightarrow V$
4 $V \rightarrow x$
5 $V \rightarrow * E$

$S' \rightarrow . S \$$	?
$S \rightarrow . V = E$	\$
$S \rightarrow . E$	\$
$V \rightarrow . x$	=
$V \rightarrow . * E$	=
$E \rightarrow . V$	\$

$\text{First}(\$): \{\$\}$

3. 继续选择E来生成新的item. 此时 β 是空串, z 是 \$, 因此 βz 是\$

例: LR(1) Closure

```
Closure( $I$ ) =  
  repeat  
    for any item ( $A \rightarrow \alpha \bullet X\beta, z$ ) in  $I$   
      for any production  $X \rightarrow \gamma$   
        for any  $w \in \text{First}(\beta z)$   
           $I \leftarrow I \cup \{(X \rightarrow \bullet \gamma, w)\}$   
    until  $I$  does not change  
  return  $I$ 
```

0 $S' \rightarrow S \$$
1 $S \rightarrow V = E$
2 $S \rightarrow E$
3 $E \rightarrow V$
4 $V \rightarrow x$
5 $V \rightarrow * E$

$S' \rightarrow . S \$$	?
$S \rightarrow . V = E$	\$
$S \rightarrow . E$	\$
$V \rightarrow . x$	=
$V \rightarrow . * E$	=
$E \rightarrow . V$	\$
$V \rightarrow . x$	\$
$V \rightarrow . * E$	\$



$\text{First}(\$): \{\$ \}$

4. 继续选择V生成新的item. 此时 β 相当于空串, z 是 \$, 因此 βz 是\$

例: LR(1) Closure

```
Closure(I) =  
  repeat  
    for any item ( $A \rightarrow \alpha \bullet X\beta, z$ ) in I  
      for any production  $X \rightarrow \gamma$   
        for any  $w \in \text{First}(\beta z)$   
           $I \leftarrow I \cup \{(X \rightarrow \bullet \gamma, w)\}$   
    until I does not change  
  return I
```

0 $S' \rightarrow S \$$
1 $S \rightarrow V = E$
2 $S \rightarrow E$
3 $E \rightarrow V$
4 $V \rightarrow x$
5 $V \rightarrow * E$

$S' \rightarrow . S \$$	?
$S \rightarrow . V = E$	\$
$S \rightarrow . E$	\$
$V \rightarrow . x$	=
$V \rightarrow . * E$	=
$E \rightarrow . V$	\$
$V \rightarrow . x$	\$
$V \rightarrow . * E$	\$

观察: 有些项比较相似, 可以有更compact的等价表示!

例: LR(1) Closure

```

Closure(I) =
  repeat
    for any item ( $A \rightarrow \alpha \bullet X\beta, z$ ) in  $I$ 
      for any production  $X \rightarrow \gamma$ 
        for any  $w \in \text{First}(\beta z)$ 
           $I \leftarrow I \cup \{(X \rightarrow \bullet \gamma, w)\}$ 
    until  $I$  does not change
  return  $I$ 

```

0 $S' \rightarrow S \$$
 1 $S \rightarrow V = E$
 2 $S \rightarrow E$
 3 $E \rightarrow V$
 4 $V \rightarrow x$
 5 $V \rightarrow * E$

$S' \rightarrow . S \$$	?
$S \rightarrow . V = E$	\$
$S \rightarrow . E$	\$
$V \rightarrow . x$	=
$V \rightarrow . * E$	=
$E \rightarrow . V$	\$
$V \rightarrow . x$	\$
$V \rightarrow . * E$	\$



$S' \rightarrow . S \$$	?
$S \rightarrow . V = E$	\$
$S \rightarrow . E$	\$
$V \rightarrow . x$	\$, =
$V \rightarrow . * E$	\$, =
$E \rightarrow . V$	\$

观察: 有些项比较相似, 可以有更compact的等价表示!

LR(1) Parsing: Goto

- 和LR(0)项集的Goto算法基本相同

Goto(I, X) = LR(1)
 $J \leftarrow \{\}$
for any item $(A \rightarrow \alpha \bullet X\beta, \textcolor{blue}{z})$ in I
 add $(A \rightarrow \alpha X \bullet \beta, \textcolor{blue}{z})$ to J
return **Closure**(J)

Goto(I, X) = LR(0)
set J to the empty set
for any item $A \rightarrow \alpha \bullet X\beta$ in I
 add $A \rightarrow \alpha X \bullet \beta$ to J
return **Closure**(J)

Only change the notion of item!

LR(1)的Reduce Action

```
 $R \leftarrow \{ \}$   
for each state  $I$  in  $T$   
  for each item  $(A \rightarrow \alpha \bullet, z)$  in  $I$   
     $R \leftarrow R \cup \{(I, z, A \rightarrow \alpha)\}$ 
```

LR(1)

```
 $R \leftarrow \{ \}$   
for each state  $I$  in  $T$   
  for each item  $A \rightarrow \alpha \bullet$  in  $I$   
    for each token  $X$  in  $\text{Follow}(A)$   
       $R \leftarrow R \cup \{(I, X, A \rightarrow \alpha)\}$ 
```

SLR(1)

Add z in items and reduce actions

- The action $(I, z, A \rightarrow \alpha)$ indicates that in state I , on lookahead symbol z , the parser will reduce by rule $A \rightarrow \alpha$.
 - The lookahead token of LR(1) is more accurate than that of SLR.
 - Some tokens in $\text{Follow}(A)$ may not always be valid in the states with $A \rightarrow \alpha$.

如果要根据 $A \rightarrow \alpha\beta \bullet$, **a** 归约, 下一个输入符号**必须是**a

Comparison of LR(0), SLR(1), and LR(1)

- When can we reduce?
 - **LR(0)**: if the state has a completed item (i.e., dot at the end, $A \rightarrow \alpha \bullet$)
 - **SLR(1)**: only if the next input token is in the Follow set of A, where $A \rightarrow \alpha$ is the production to use
 - **LR(1)**: only if the next input token is exactly a terminal or \$ (explicitly specified in the item)

方法	归约条件	精度
LR(0)	• 在末尾即可归约（无需看输入）	最低
SLR(1)	下一个 token $\in \text{Follow}(A)$ （全局集合）	中等
LR(1)	下一个 token 恰为项中的 a （精确）	最高

LR(1) Parsing的局限

- LR(1) parsing tables can be very large, with many states.

0 $S' \rightarrow S \$$

1 $S \rightarrow V = E$

2 $S \rightarrow E$

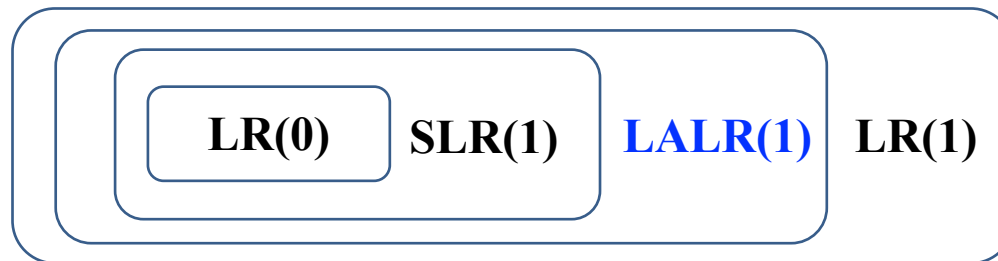
3 $E \rightarrow V$

4 $V \rightarrow x$

5 $V \rightarrow * E$

	x	*	=	\$	S	E	V
1	s8	s6			g2	g5	g3
2				a			
3			s4	r3			
4	s11	s13				g9	g7
5				r2			
6	s8	s6				g10	g12
7				r3			
8			r4	r4			
9				r1			
10			r5	r5			
11				r4			
12			r3	r3			
13	s11	s13				g14	g7
14				r5			

(a) LR(1)



5. LALR(1)

一般LALR指的就是LALR(1)，类似SLR指SLR(1)

The LALR(1) Approach

- Two main approaches to building LALR(1) parsers:
 1. Build the full LR(1) automaton, then merge states with identical cores.
 - ~~2. Build an LR(0) automaton and compute lookaheads for each state (more efficient?).~~
- **We'll focus on the first approach for clarity.**

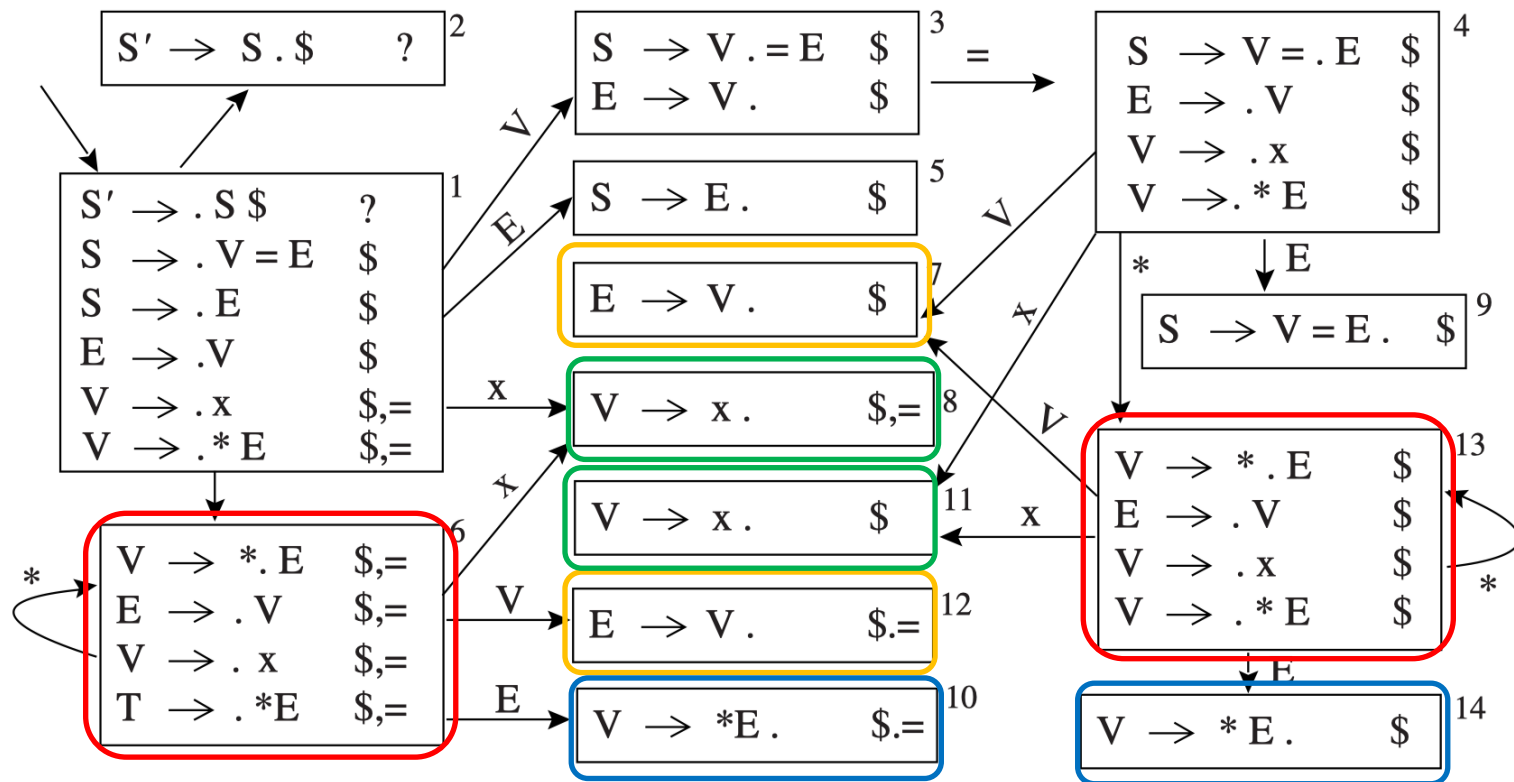
- 0 $S' \rightarrow S \$$
- 1 $S \rightarrow V = E$
- 2 $S \rightarrow E$
- 3 $E \rightarrow V$
- 4 $V \rightarrow x$
- 5 $V \rightarrow * E$

- 0 $S' \rightarrow S \$$
- 1 $S \rightarrow V = E$
- 2 $S \rightarrow E$
- 3 $E \rightarrow V$
- 4 $V \rightarrow x$
- 5 $V \rightarrow * E$



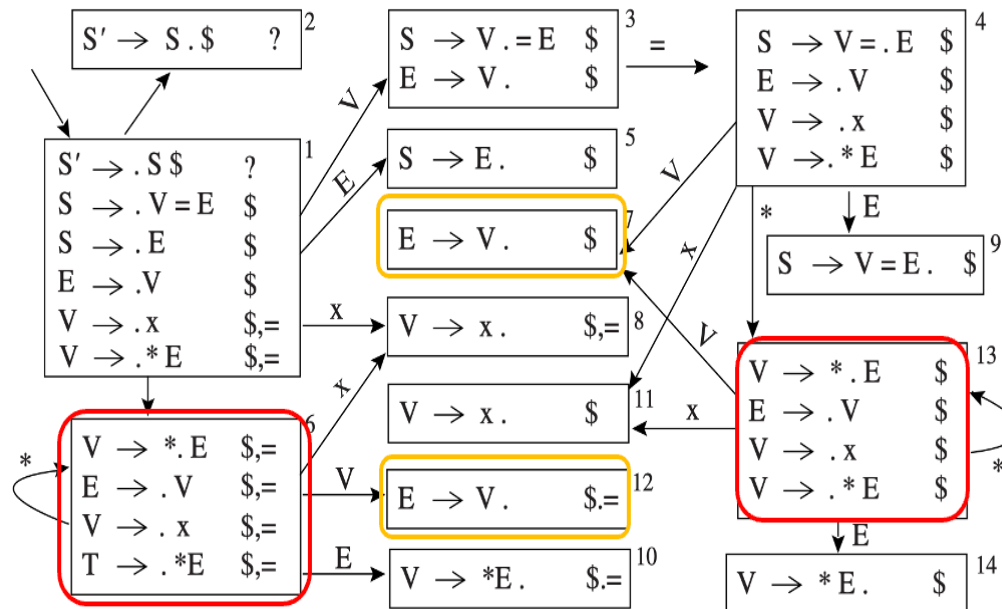
LALR(1) Parsing in a Nutshell

- **Idea:** merging states whose items are identical except for lookahead sets in the LR(1) parsing table.
- How to merge?



The Core of a Set of LR(1) Items

- **Definition:** The **core** of a set of LR items is the set of first components (Without the lookahead terminals)
 - E.g.,, the core of $\{ X \rightarrow a \bullet b \quad b, Y \rightarrow g \bullet d \quad d \}$ is $\{ X \rightarrow a \bullet b, Y \rightarrow g \bullet d \}$
- In LR(1), may states have the same core

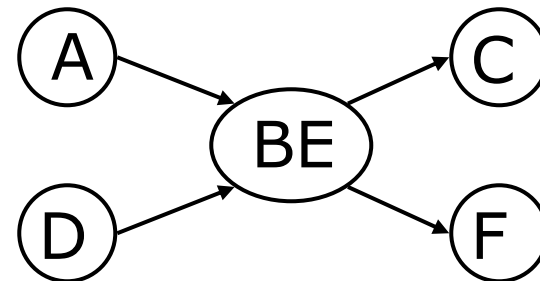
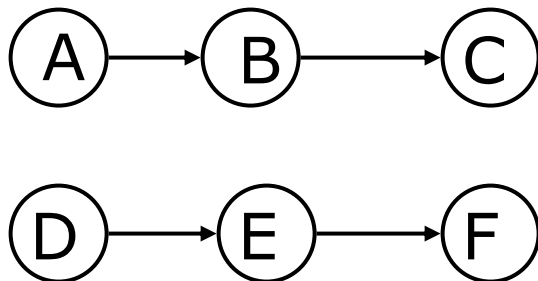


Building LALR(1) States by Merging

- Consider for example the LR(1) states
$$\{[X \rightarrow \alpha \bullet \ a], [Y \rightarrow \beta \bullet \ c]\}$$
$$\{[X \rightarrow \alpha \bullet \ b], [Y \rightarrow \beta \bullet \ d]\}$$
- They have the same core and can be merged, and the merged state contains:
$$\{[X \rightarrow \alpha \bullet \ a, b], [Y \rightarrow \beta \bullet \ c, d]\}$$
- These are called LALR(1) states
 - Stands for LookAhead LR
 - Typically, 10x fewer LALR(1) states than LR(1)

From LR(1) to LALR(1)

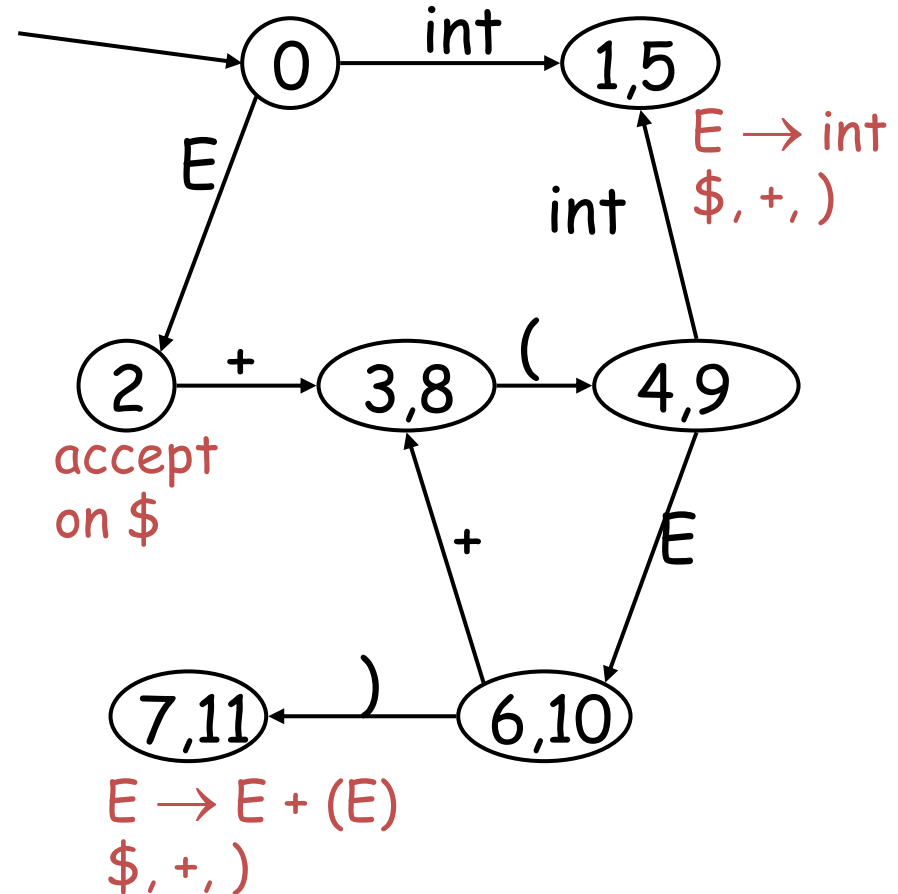
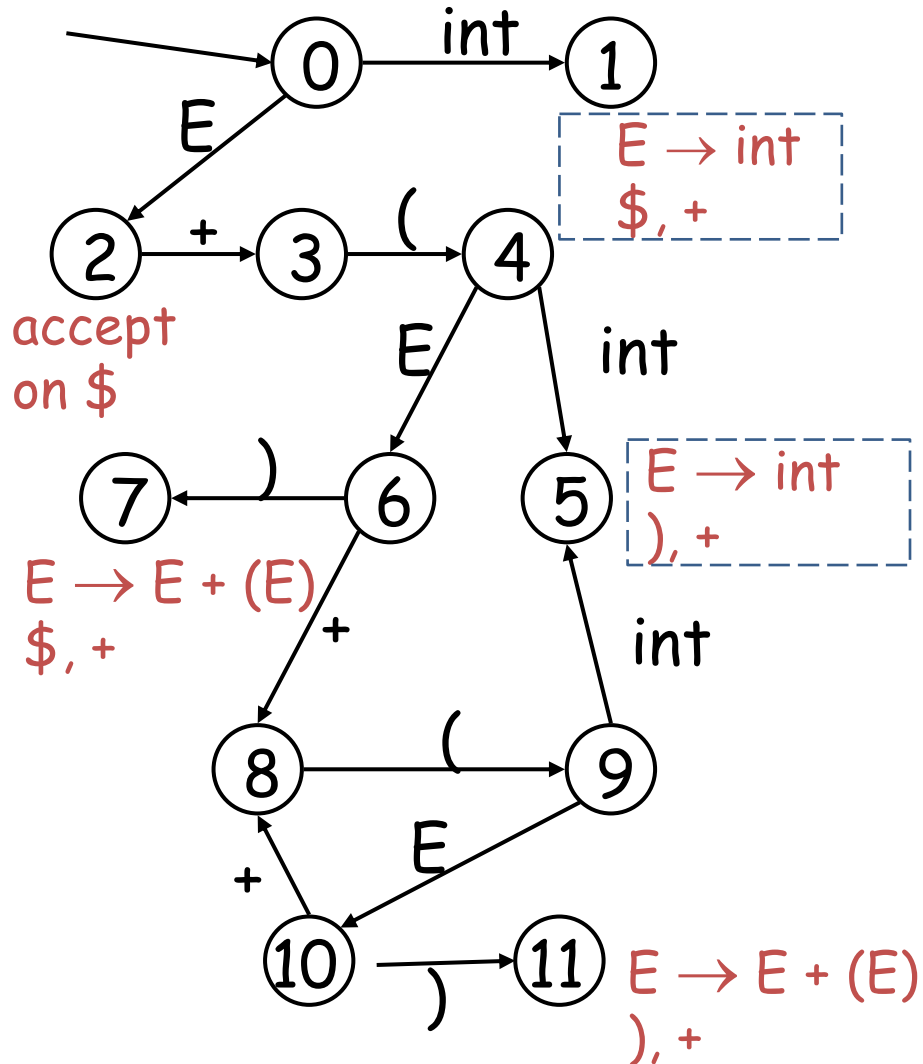
- **Repeat until all states have distinct core**
 1. Choose two distinct states with same core
 2. Merge the states by creating a new one with the union of all the items
 3. Point edges from predecessors to new state; new state points to all the previous successors



- **Merge items with common cores**
- **Change GOTO table to reflect merges**

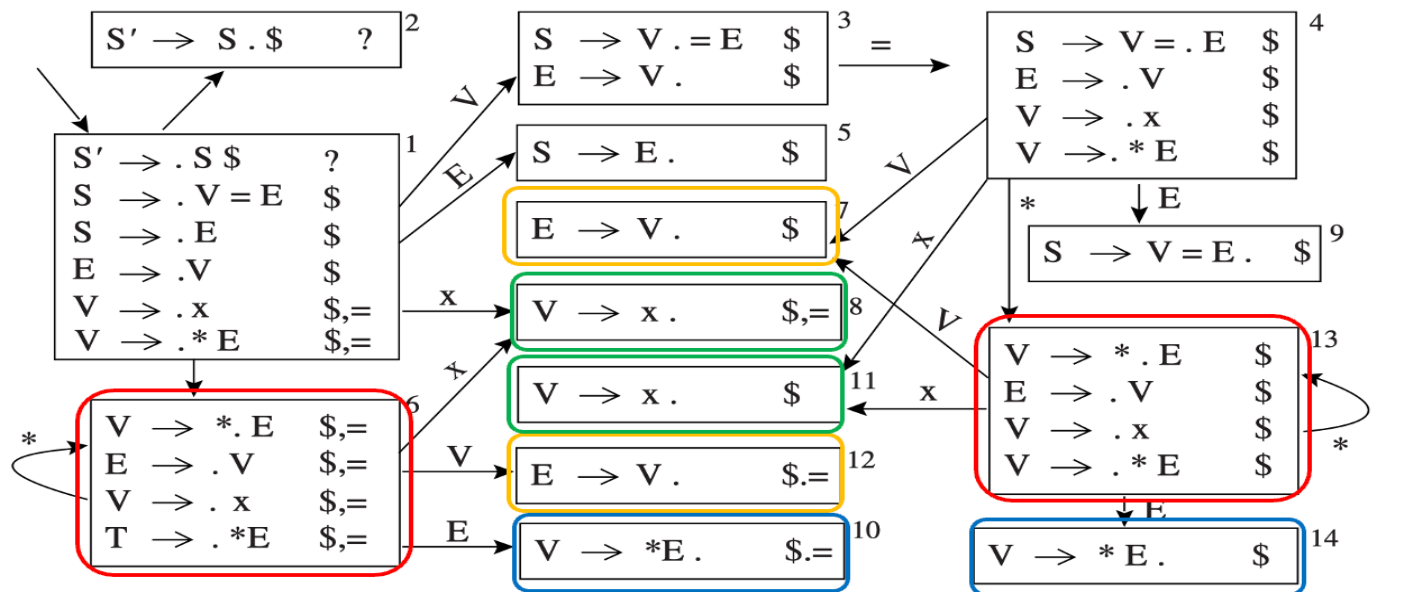
Example: LR(1) to LALR(1) DFA

假设状态1, 5合并(Core是 $E \rightarrow \text{Int}$, Lookahead sets取并集); 状态4, 9合并



练习: 从LR(1) 到LALR(1)

0 $S' \rightarrow S \$$
 1 $S \rightarrow V = E$
 2 $S \rightarrow E$
 3 $E \rightarrow V$
 4 $V \rightarrow x$
 5 $V \rightarrow * E$



	x	*	=	\$	S	E	V
1	s8	s6			g2	g5	g3
2				a			
3			s4	r3			
4	s11	s13				g9	g7
5			r2				
6	s8	s6				g10	g12
7			r3				
8			r4	r4			
9			r1				
10			r5	r5			
11			r4				
12			r3	r3			
13	s11	s13				g14	g7
14			r5				



	x	*	=	\$	S	E	V
1	s8	s6			g2	g5	g3
2				a			
3			s4	r3			
4	s8	s6				g9	g7
5			r2				
6	s8	s6				g10	g7
7			r3	r3			
8			r4	r4			
9			r1				
10			r5	r5			

(a) LR(1)

(b) LALR(1)

LALR(1) vs. LR(1) Parsing

- Consider the following two LR(1) states

$\{[X \rightarrow \alpha \bullet \ a], [Y \rightarrow \beta \bullet \ b]\}$

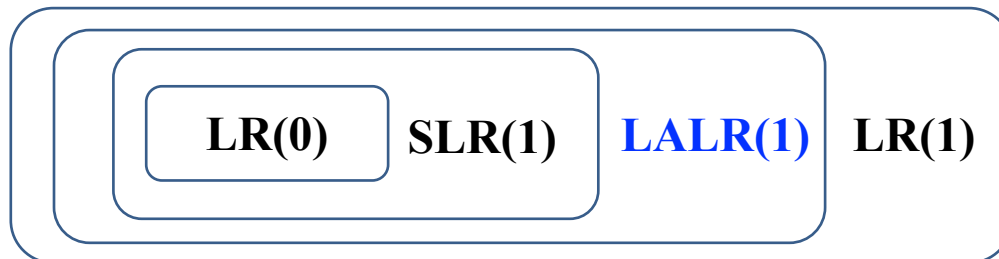
$\{[X \rightarrow \alpha \bullet \ b], [Y \rightarrow \beta \bullet \ a]\}$

- And the merged LALR(1) state is

$\{[X \rightarrow \alpha \bullet \ a, b], [Y \rightarrow \beta \bullet \ a, b]\}$

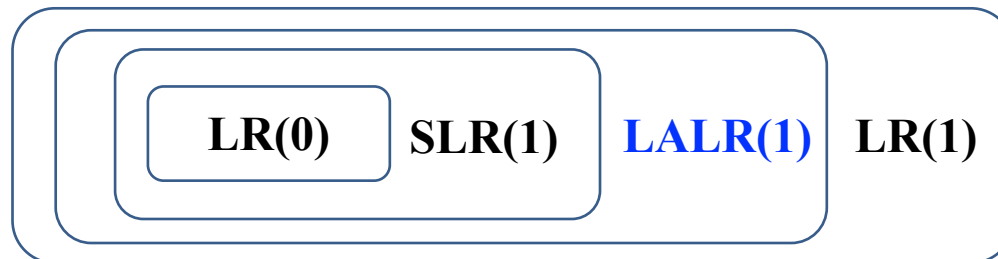
- Has a new reduce-reduce conflict!** (why?)
- In practice such cases are rare

(LALR表达力足够强大)



LALR(1) vs. LR(1) Parsing

- **LR(1)**
 - 把期望的向前看符号也加入项中：LR(1)项
 - 向前看符号(串)的长度即为LR(k)中的 k
 - 充分利用向前看符号，但是状态很多
- **LALR(1)**
 - 介于SLR(1)和LR(1)之间，分析表和SLR一样大
 - LALR已经可以处理大部分的程序设计语言

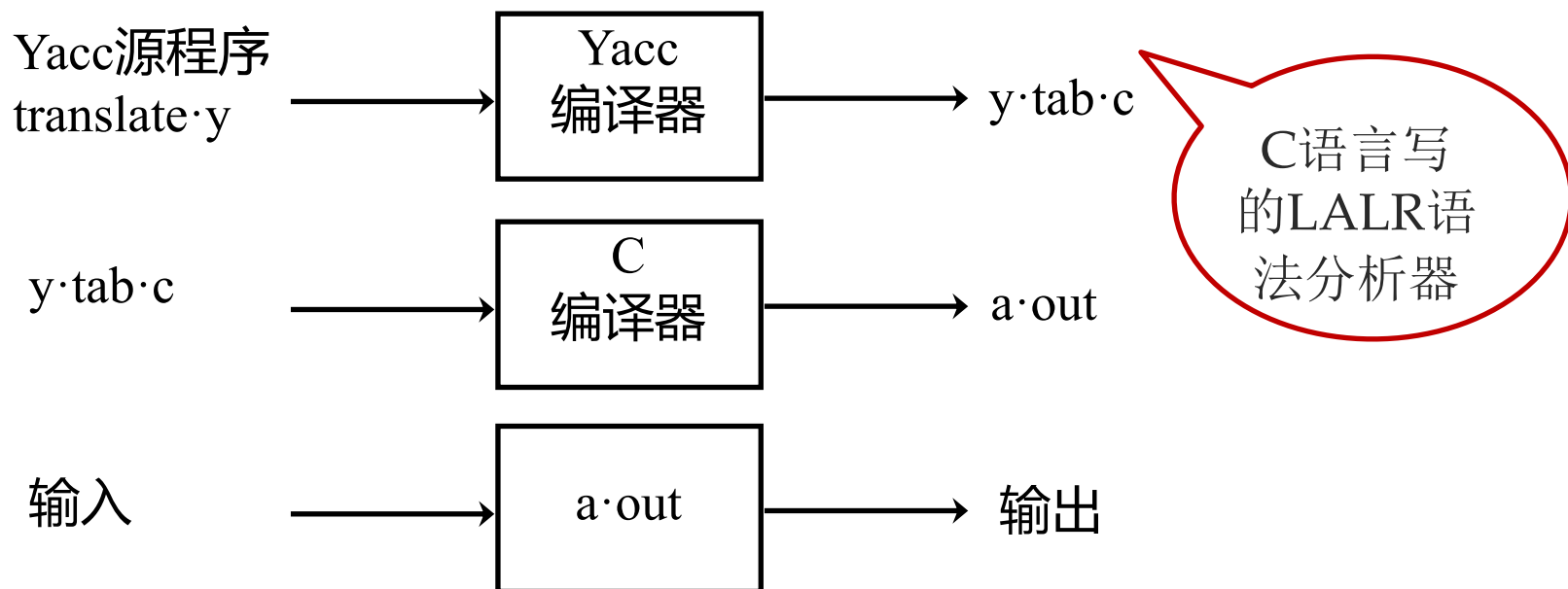


3. Parser Generator

语法分析器的生成器Yacc

- **Yacc : yet another compiler-compiler**
 - 基于LALR(1), 用BNF(Backus Naur Form)形式书写
- **Yacc 的 GNU 版叫做 Bison**

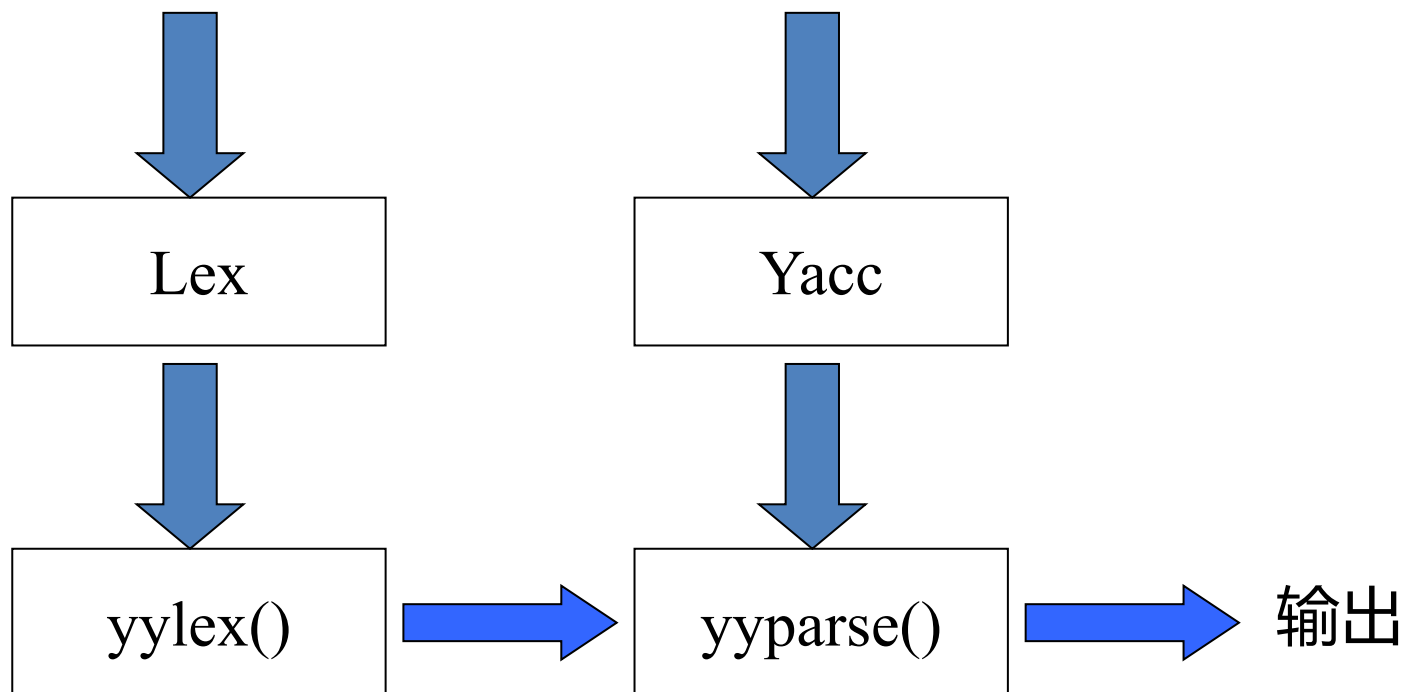
yacc <options> <filename ending with .y>



Lex和Yacc的联系

词法规则(.l文件)

语法规则(.y文件)



YACC源程序的结构

- **声明**

- 放置C声明和对词法单元的声明

- **翻译规则**

- 指明产生式及相关的语义动作

声明

%%

翻译规则

%%

辅助性C语言例程

- **辅助性C语言例程**

- 被直接拷贝到生成的C语言源程序中
- 可在语义动作中调用
- 包括yylex()，这个函数返回词法单元，
可以由Lex生成

例: Yacc使用

$exp \rightarrow exp \text{ addop } term \mid term$
 $addop \rightarrow + \mid -$
 $term \rightarrow term \text{ mulop } factor \mid factor$
 $mulop \rightarrow *$
 $factor \rightarrow (exp) \mid number$

Rules:

- Rule {Action Code}
- Action code will be executed after the parse performs a reduce action using the rule.
语义动作: $$$ = \$1 + \$3$

```
%{  
#include <stdio.h>  
#include <ctype.h>  
int yylex(void);  
int yyerror (char * s);  
%}  
%token NUMBER  
%%  
command: exp {printf("%d\n", $1);};  
exp: exp '+' term {$$ = $1 + $3;}  
    | exp '-' term {$$ = $1 - $3;}  
    | term {$$ = $1}  
;  
term: term '*' factor {$$ = $1 * $3;}  
    | factor {$$ = $1;}  
;  
factor: NUMBER {$$ = $1;}  
       | '(' exp ')' {$$ = $2;}  
;
```

$$$$ 表示和产生式头相关的属性值, $\$i$ 表示
产生式体中第 i 个文法符号的属性值

Yacc文件格式中的几个问题

- **消除二义性:** 为算符指定优先级与结合律

`%left '-' '+'`

`%left '*' '/'`

`%right UMINUS` `/* negation--unary minus */`

`%right '^'` `/* exponentiation */`

- **冲突解决**

- 归约/归约冲突：选择Yacc说明中先出现的产生式

$A \rightarrow \alpha.$ $B \rightarrow \alpha.$

- 移进/归约冲突：**移近优先**

$I_4: S \rightarrow iS \cdot eS$

$S \rightarrow iS \cdot$

更通用的方法: 改写文法以消除冲突
(例如, 消除二义性的同时也可能减少了冲突)

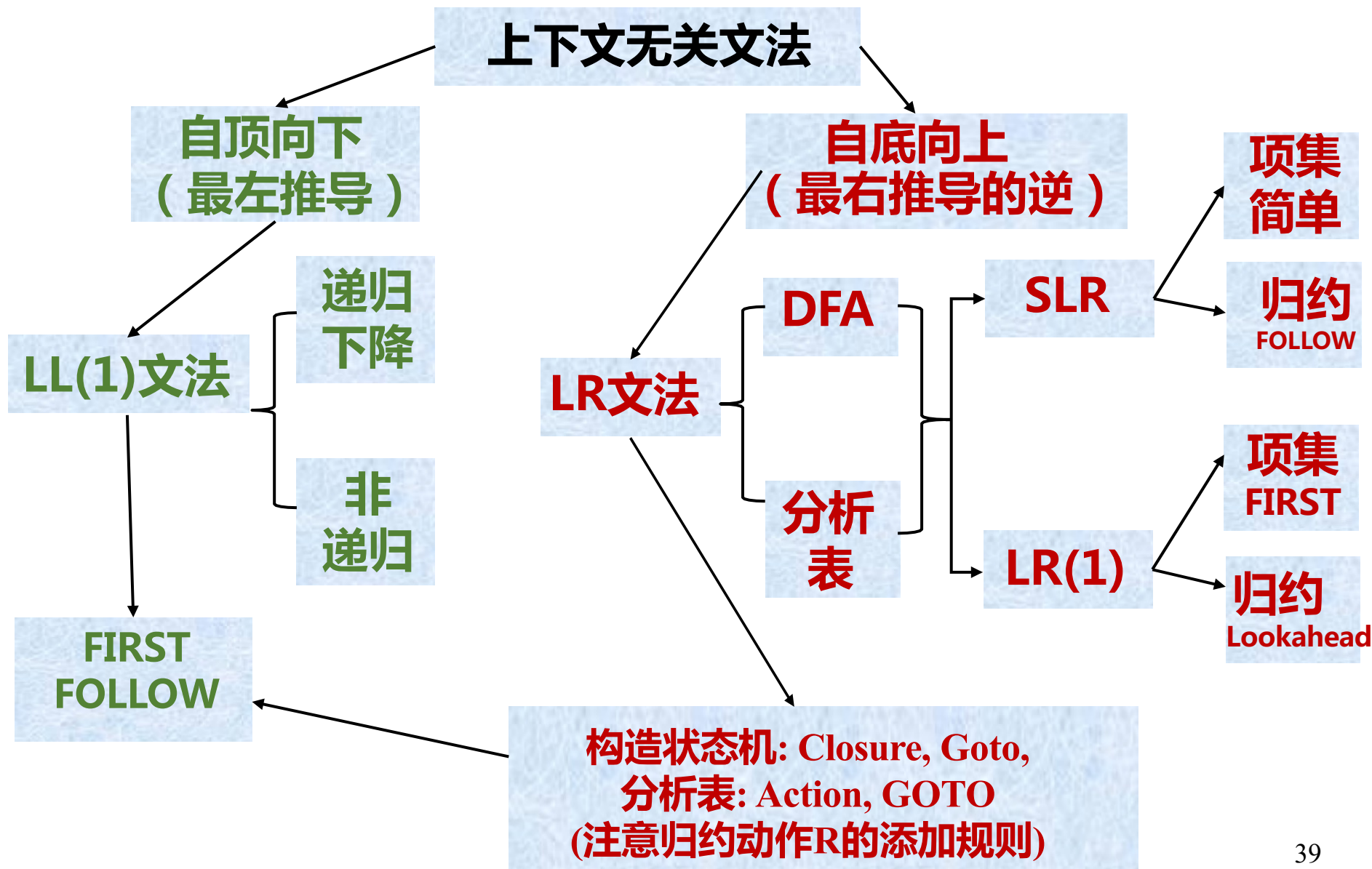
4. 语法分析小结

LL(1)和LR(1)分析对比

	LL(1)方法	LR(1)方法
建立分析树	自顶而下	自底而上
归约or推导	最左推导	规范归约(最右推导的逆)
分析表	非终结符×终结符，小	状态×文法符号，大
分析栈	文法符号栈	状态栈，信息更多
向前看机制	First/Follow 决定展开哪条产生式	识别完整 α 后，看 1 个符号决定归约
处理左递归	不能直接处理	可以处理

- LL(1): 向前看下一个输入根据First, Follow确定使用哪条产生式推导
$$A \rightarrow \alpha_1 \mid \alpha_2 \mid \alpha_3$$
- LR(1): 在识别出整个 α 后，再往前看1个符号，然后确定使用哪条产生式归约
$$A \rightarrow \alpha \quad B \rightarrow \alpha$$

LL(1), SLR(1)和LR(1)对比



LR(0), SLR(1), LR(1)和LALR(1)对比

Feature	LR(0)	SLR(1)	LALR(1)	LR(1)
Full Name	LR(0) Parser	Simple LR(1) Parser	Look-Ahead LR(1)	Canonical LR(1) Parser
Lookahead Tokens	None (0 tokens)	1 token (via FOLLOW sets)	1 token (computed more precisely)	1 token (exact)
Parsing Power	Weakest	Stronger than LR(0)	Stronger than SLR(1), almost as powerful as LR(1)	Most powerful
Grammar Class	LR(0) grammars	SLR(1) grammars	LALR(1) grammars	LR(1) grammars
State Generation	Based on LR(0) items (e.g., $A \rightarrow \alpha \bullet \beta$)	Based on LR(0) items with FOLLOW set to resolve conflicts	Merges compatible LR(1) states	Based on full LR(1) items (e.g., $[A \rightarrow \alpha \bullet \beta, a]$)
Parser Size (states)	Smallest	Same as LR(0)	Moderately larger than SLR	Largest (can grow exponentially)
Handling of Shift/Reduce Conflicts	Poor - requires grammar modification	Can resolve some with FOLLOW sets	> SLR, can handle most practical cases	Best - can handle any LR(1) grammar
Pros	- Simplest to understand - Smallest parser tables - Fast parser generation	- Relatively simple implementation - Same number of states as LR(0) - Better handling of conflicts than LR(0)	- Good balance of power and efficiency - Handles most practical programming languages - Reasonable table size	- Maximum parsing power - Can handle any LR(1) grammar - Most precise error detection
Cons	- Cannot handle many useful grammars - Many shift/reduce & reduce/reduce conflicts	- Still has conflicts on many useful grammars - FOLLOW set approximation causes some false conflicts	- More complex implementation than SLR - Occasional conflicts that LR(1) would resolve	- Large parser tables - Slow parser generation - High memory consumption - Complex imple

练习题

1. LR(0)在实际中很少使用，为什么？

不进行任何展望(lookahead)，只利用栈顶信息，极易发生冲突

2. SLR(1)是怎么在LR(0)基础上改进的？

展望(lookahead)一个输入token：归约时使用Follow集

3. 怎么理解LR分析中的闭包 (Closure) 操作？

归类“期待”意义上等价的项目，对非终结符结合产生式不断添加新的项目

4. Top-down和Bottom-up哪个能覆盖更多的文法？简要解释。

Bottom-up，可以处理左递归和共同前缀

5. LR(0)解析表(Parse Table)存放的内容是什么？

Action和Goto，Action是对终结符的动作表，Goto是非终结符的跳转表

6. $A \rightarrow BC \cdot$ 是什么意思？

LR中的一个完成项目，在解析进展上，已经识别了BC生成的token，可以使用 $A \rightarrow BC$ 进行归约



Thank you all for your attention

5. 错误恢复

- **Local Error Recovery**
- **Global Error Recovery**

Error Recovery

- What should happen when your parser finds an error in the user's input?
 1. stop immediately and signal an error
 2. record the error but try to continue
- In the first case, the user must recompile from scratch after possibly a trivial fix
- In the second case, the user might be overwhelmed by a whole series of error messages, all caused by essentially the same problem

Error Recovery

- **Error recovery**
 - process of adjusting input stream so that the parser can continue after unexpected input
- **Possible adjustments**
 - delete tokens
 - insert tokens
 - substitute tokens
- **Classes of recovery**
 - **local recovery**: adjust input at the point where error was detected (and also possibly immediately after)
 - **global recovery**: adjust input before point where error was detected.

Local Error Recovery

- 基本策略：寻找"同步符号"（**Synchronizing Token**）
- 在底部向上解析器中，通过向文法添加**错误产生式**来实现：

$$\begin{aligned} \text{exp} &\rightarrow \text{NUM} \\ &| \text{exp PLUS exp} \\ &| (\text{exp}) \end{aligned}$$
$$\begin{aligned} \text{exps} &\rightarrow \text{exp} \\ &| \text{exps ; exp} \end{aligned}$$

- General strategy for both bottom-up and top-down:
 - look for a **synchronizing token**
- Accomplished in bottom-up parsers by adding error rules to grammar:

$$\begin{aligned} \text{exp} &\rightarrow (\text{error}) // \text{括号内的错误} \\ \text{exps} &\rightarrow \text{exp} \\ &| \text{error ; exp} // \text{语句级错误} \end{aligned}$$

Local Error Recovery

- In general, follow error with a synchronizing token.
- Recovery steps:
 1. **Pop stack** (if necessary) until a state is reached in which the action for the error token is shift
 2. **Shift** the error token
 3. **Discard input** symbols (if necessary) until a state is reached that has a non-error action
 4. **Resume** normal parsing

关键思想：利用 **error** 作为特殊 token，让解析器从错误中“跳出”，找到下一个合法的恢复点（同步符号如 ; 或)）继续解析。

Example: Local Error Recovery

$\text{exp} \rightarrow \text{NUM}$
 $\quad | \text{exp PLUS exp}$
 $\quad | (\text{exp})$

$\text{exps} \rightarrow \text{exp}$
 $\quad | \text{exps ; exp}$

$\text{exp} \rightarrow (\text{error})$
 $\text{exps} \rightarrow \text{exp}$
 $\quad | \text{error ; exp}$

input: NUM PLUS (NUM PLUS @# \$ PLUS NUM) PLUS NUM

stack: exp PLUS (exp PLUS

yet to read

@#\$ is an unexpected token!

Example: Local Error Recovery

$\text{exp} \rightarrow \text{NUM}$
 $\quad | \text{exp PLUS exp}$
 $\quad | (\text{exp})$

$\text{exps} \rightarrow \text{exp}$
 $\quad | \text{exps ; exp}$

$\text{exp} \rightarrow (\text{error})$
 $\text{exps} \rightarrow \text{exp}$
 $\quad | \text{error ; exp}$

input: NUM PLUS (NUM PLUS @#\$ PLUS NUM) PLUS NUM

stack: exp PLUS (

yet to read

pop stack until shifting “error” can result in correct parse

Example: Local Error Recovery

$\text{exp} \rightarrow \text{NUM}$
 $\quad | \text{exp PLUS exp}$
 $\quad | (\text{exp})$

$\text{exps} \rightarrow \text{exp}$
 $\quad | \text{exps ; exp}$

$\text{exp} \rightarrow (\text{error})$
 $\text{exps} \rightarrow \text{exp}$
 $\quad | \text{error ; exp}$

input:

NUM PLUS (NUM PLUS @#\$ PLUS NUM) PLUS NUM

yet to read

stack:

exp PLUS (error

shift “error”

Example: Local Error Recovery

$\text{exp} \rightarrow \text{NUM}$
 $\quad | \text{exp PLUS exp}$
 $\quad | (\text{exp})$

$\text{exps} \rightarrow \text{exp}$
 $\quad | \text{exps ; exp}$

$\text{exp} \rightarrow (\text{error})$
 $\text{exps} \rightarrow \text{exp}$
 $\quad | \text{error ; exp}$

input:

NUM PLUS (NUM PLUS @#\$ PLUS NUM) PLUS NUM

yet to read

stack:

exp PLUS (error

discard input until we can legally shift or reduce

Example: Local Error Recovery

$\text{exp} \rightarrow \text{NUM}$
 $\quad | \text{exp PLUS exp}$
 $\quad | (\text{exp})$

$\text{exps} \rightarrow \text{exp}$
 $\quad | \text{exps ; exp}$

$\text{exp} \rightarrow (\text{error})$
 $\text{exps} \rightarrow \text{exp}$
 $\quad | \text{error ; exp}$

input:

NUM PLUS (NUM PLUS @#\$ PLUS NUM) PLUS NUM

yet to read

stack:

exp PLUS (error)

SHIFT)

Example: Local Error Recovery

$\text{exp} \rightarrow \text{NUM}$
 $\quad | \text{exp PLUS exp}$
 $\quad | (\text{exp})$

$\text{exps} \rightarrow \text{exp}$
 $\quad | \text{exps} ; \text{exp}$

$\text{exp} \rightarrow (\text{error})$
 $\text{exps} \rightarrow \text{exp}$
 $\quad | \text{error} ; \text{exp}$

input:

NUM PLUS (NUM PLUS @#\$ PLUS NUM) PLUS NUM

yet to read

stack:

exp PLUS exp

REDUCE using $\text{exp} ::= (\text{error})$

Example: Local Error Recovery

$\text{exp} \rightarrow \text{NUM}$
 $\quad | \text{exp PLUS exp}$
 $\quad | (\text{exp})$

$\text{exps} \rightarrow \text{exp}$
 $\quad | \text{exps} ; \text{exp}$

$\text{exp} \rightarrow (\text{error})$
 $\text{exps} \rightarrow \text{exp}$
 $\quad | \text{error} ; \text{exp}$

input:

NUM PLUS (NUM PLUS @#\$ PLUS NUM) PLUS NUM

yet to read

stack:

exp PLUS exp

continue parsing...

Summary: Local Error Recovery

- 使用**错误产生式**来完成语法错误恢复
 - 错误产生式 $A \rightarrow \text{error } \alpha$
 - 例如： $\text{stmt} \rightarrow \text{error};$
- 定义哪些非终结符号有错误恢复动作
 - 比如：表达式、语句、块、函数定义等非终结符号
- 当语法分析器遇到错误时
 - 不断弹出栈中状态，直到栈顶状态包含项 $A \rightarrow \cdot \text{error } \alpha$
 - 分析器将error移入栈中
 - 如果 α 为空，分析器直接执行归约，并调用相关的语义动作；否则**跳过**一些符号，找到可以归约为 α 的串为止

4. 错误恢复

- Local Error Recovery
- Global Error Recovery

Global Error Recovery

- Global error recovery
 - Determines the smallest set of insertions, deletions or replacements that will allow a correct parse
 - Even if those operations occur before an error would have been detected

Burke-Fisher Error Recovery

- Global error recovery
 - Determines the smallest set of insertions, deletions or replacements that will allow a correct parse,
 - Even if those operations occur before an error would have been detected
- Burke-Fisher error repair
 - tries every possible single-token insertion, deletion or replacement at every point in the input up to K tokens before the error is detected
 - eg: $K = 20$; parser gets stuck at token 500; all possible repairs between token 480-500 tried
 - best repair = longest successful parse

Global Error Recovery

- Parser must be able to back up K tokens and reparse
- Mechanics:
 - parser maintains old stack and new stack

K-token window maintained in queue by parser

input: ID := NUM ; ID := ID + (ID := NUM + ...

new stack: S ; ID := ID + (

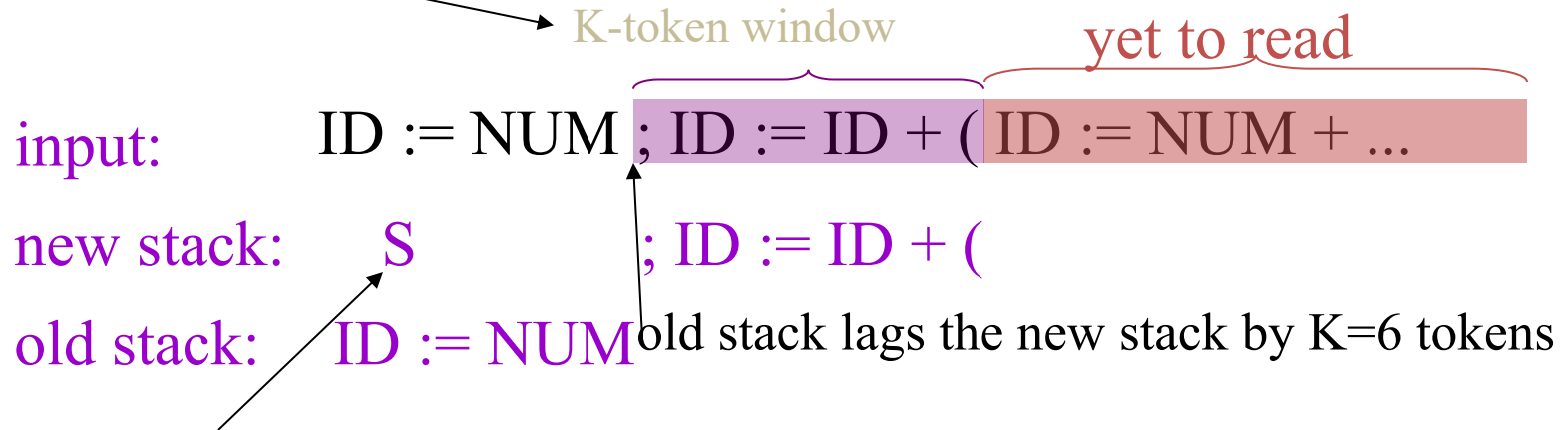
old stack: ID := NUM

Diagram details: An arrow points from "K-token window" to the input string. A purple bracket under "ID := ID + (" is labeled "K-token window". A red bracket over "ID := NUM + ..." is labeled "yet to read".

Global Error Recovery

- Parser must be able to back up K tokens and reparse
- Mechanics:
 - parser maintains old stack and new stack

K-token window maintained in queue by parser



Reduction ($E \rightarrow \text{NUM}$ and $S \rightarrow \text{ID} := E$) has applied to new stack already

Global Error Recovery

- Parser must be able to back up K tokens and reparse
- Mechanics:
 - parser maintains old stack and new stack

K-token window maintained in queue by parser

input: ID := NUM ; ID := ID + (ID := NUM + ...

new stack: S ; ID := ID + (

old stack: ID := E

K-token window

yet to read

Reduction ($E \rightarrow \text{NUM}$) applies to old stack now

Global Error Recovery

- Consider Burke-Fisher with
 - K-token window
 - N different token types
- Total number of repairs = $N + 2K * N$
 - deletions (K) +
 - insertions $(K + 1) * N +$
 - replacements $(K)(N-1)$
- Affordable in the uncommon case when there is an error